CS 212a – DATA STRUCTURES AND ALGORITHMS

FINAL PROJECT: PYTHON-BASED GAME DEVELOPMENT

I. Project Overview

For your final project, you will design, develop, and present a functional Python game that demonstrates your understanding of data structures and algorithms.

You may implement your project in the Python terminal or command-line interface, or you may use a graphical interface through libraries such as Pygame, Tkinter, Turtle, or other Python tools.

A simple running game implemented entirely in the Python terminal is acceptable. You are not required to create advanced graphics or a complex game. Your primary goal is to demonstrate how you can apply data structures and algorithms to develop a functional and useful application.

II. Project Objectives

Through this project, you should be able to:

1. Apply appropriate data structures to solve problems in your Python application.
2. Implement algorithms for searching, sorting, traversing, or processing data.
3. Develop a functional application or game using Python.
4. Explain your design decisions and demonstrate how your chosen data structures and algorithms work.
5. Apply proper coding practices, testing, debugging, and documentation.

III. Project Requirements

1. Programming Language and Tools

- You must develop your application primarily using Python.
- You may use Pygame, Tkinter, Turtle, or other appropriate Python libraries.
- You may use VS Code, PyCharm, IDLE, or another Python development environment.

- You may use external libraries for graphics, sound, and interface design. However, you must implement and explain the core data structures and algorithms required for your project.

2. Application Functionality

Your project must be a working application with a clear purpose and appropriate features. You may develop a terminal-based program, a text-based game, a simple running game, a graphical game, a simulation, or a management system.

3. Application of Data Structures

You must meaningfully implement at least **three (3) data structures** appropriate to your application. You may use data structures such as:

- Lists and arrays
- Stacks
- Queues
- Linked lists
- Dictionaries and hash tables
- Sets
- Trees, including binary search trees
- Heaps or priority queues
- Graphs

You must identify the data structures you used and explain why and how you used them in your application.

4. Application of Algorithms

You must meaningfully implement at least **two (2) algorithms or algorithmic techniques** discussed in class. Examples include:

- Linear search or binary search
- Sorting algorithms
- Tree traversal
- Graph traversal, such as BFS or DFS
- Shortest-path algorithms
- Recursion
- Priority-based processing
- Algorithms for game logic, movement, or problem-solving

You may use Python's built-in functions where appropriate, but you must demonstrate and explain your own implementation of the required algorithms.

5. Documentation

You must prepare project documentation containing the following:

- Project title and description
- Project objectives and intended users
- Features and functionality
- Data structures and algorithms implemented
- Explanation of how each data structure and algorithm works in your application
- Screenshots of your running application
- Testing results
- Conclusion and possible future improvements

IV. Suggested Project Categories

You may select any of the following project ideas or propose your own project, subject to instructor approval.

A. Terminal-Based Python Projects

- Simple Running Game with score tracking
- Text-Based Adventure Game
- Number Guessing Game with attempt tracking
- Quiz Game with a leaderboard
- Tic-Tac-Toe
- Hangman Word Game
- Maze Game using text characters
- Rock-Paper-Scissors Tournament
- Tower of Hanoi Puzzle
- Menu-Driven Student Record Management System

B. Python Games with a Graphical Interface

- Snake Game
- Maze-Solving Game
- Memory Card Matching Game
- Platformer Game
- Pong Game
- Simple Racing Game
- Tower Defense Game
- Pathfinding Visualization Game

You may use Pygame or other Python libraries to implement graphics, animation, sound, and keyboard-controlled gameplay. However, **using a graphical interface is optional**.

You are also free to propose another application or game, provided that it meets the project requirements and receives instructor approval.

V. Special Guidelines for Running Games

When you are developing a running game, you may implement it entirely in the Python terminal using text characters, obstacles, keyboard input, and a score counter. You do not need to create advanced graphics, animations, sound effects, or use external libraries.

Your running game may include the following features:

- A player character that moves or jumps over obstacles.
- Randomly generated or predefined obstacles.
- Collision detection and game-over conditions.
- Score tracking based on survival time, distance, or obstacles avoided.
- A restart option or increasing difficulty.
- A high-score record or leaderboard.

You must incorporate appropriate data structures and algorithms into your game. For example, you may use lists to store obstacles, dictionaries to store player statistics, queues to manage game events, or sorting algorithms to rank high scores.

The features listed above are suggestions rather than mandatory requirements. You should choose features and data structures that are appropriate to your project.

Remember: A simple terminal-based running game is an acceptable final project, provided that it works correctly, demonstrates your application of the required data structures and algorithms, and allows you to explain your implementation during your project defense.

VI. Submission Requirements

You must submit the following requirements:

1. **Python Source Code:** Submit all files needed to run your application, including relevant assets and supporting files.
2. **Project Documentation:** Submit your report explaining your application's purpose, features, data structures, algorithms, and testing results.
3. **README File:** Include instructions for installing dependencies and running your application.
4. **Demonstration:** Present a live or recorded demonstration of your working application.
5. **Presentation and Defense:** Explain your implementation, demonstrate the required data structures and algorithms, and answer questions about your source code.

You must acknowledge any external libraries, tutorials, code references, and assets you use. You must also be able to explain your code and demonstrate your understanding of your implementation during the project defense.

VII. Project Presentation and Defense

During your final presentation, you must:

1. Introduce your project and explain the problem it addresses or the purpose it serves.
2. Demonstrate your application's major features.
3. Identify and explain the data structures you implemented.
4. Demonstrate and explain the algorithms you implemented.
5. Discuss the challenges you encountered and how you addressed them.
6. Answer questions about your source code, design decisions, and program behavior.

You may be asked to trace an algorithm, explain a particular section of your code, or modify a minor feature to demonstrate your understanding of your implementation.

VIII. Grading Criteria

Your final project will be evaluated using the following criteria:

Criteria	Weight
Application of Data Structures	25%
Application of Algorithms	20%
Functionality and Correctness	20%
Code Quality and Organization	10%
User Interface and Creativity	10%
Documentation	5%
Presentation and Defense	10%

Total	**100%**

IX. General Guidelines

1. You must obtain instructor approval for your proposed project before beginning full implementation.
2. You must ensure that your project scope is achievable within the allotted development period.
3. You must follow the instructor's instructions regarding individual or group work.
4. You must ensure that your selected data structures and algorithms serve a clear purpose in your application.
5. Your project will be evaluated based on implementation, correctness, and your understanding of data structures and algorithms, not solely on visual appearance.
6. You must ensure that your application runs in the designated testing environment or provide clear setup instructions.
7. You must submit original work, acknowledge external resources, and be prepared to explain the code you submit.

X. Expected Output

By the end of the project, you must deliver a functional Python application or game that demonstrates your understanding of data structures and algorithms through a practical, documented, and explainable implementation.

Your project may be simple or visually advanced. What matters most is how effectively you apply data structures and algorithms to make your application work.